```
1 runs, 1 assertions, 0 failures, 0 errors, 0 skips
```

The method under test thinks it's writing to a file, but instead it's just appending to a string. Actually, even that statement is a little strong—the method under test thinks it's sending the message << to an object that will receive it and do something. Even though the method may have been written with a file in mind, any object that responds to << will work. At the end, we can then just test that the content is correct given what the receiver does with the message <<.

We didn't have to use a string; an array would work just as well for the purposes of the test:

ducktyping/test_add_customer_3.rb

```
require "minitest/autorun"
require_relative "add_customer"

class TestAddCustomer < Minitest::Test
  def test_add
    customer = Customer.new("Ima", "Customer")
    fake_file = []
    customer.append_name_to_file(fake_file)
    assert_equal(["Ima", " ", "Customer"], fake_file)
  end
end
```

produces:

```
Run options: --seed 47726
# Running:

.
Finished in 0.000415s, 2409.6385 runs/s, 2409.6385 assertions/s.
1 runs, 1 assertions, 0 failures, 0 errors, 0 skips
```

Indeed, this form may be more convenient if we wanted to check that the correct individual elements were inserted.

You could at this point, make the argument that the original method append_name_to_file is misnamed and that the method should be just append_name—especially because the argument is also named file. We can see an argument either way on this point. On the one hand, file is only a limited subset of what this method actually works on, on the other hand putting file in the method name signals to users of the method what's expected. The point here is to make your intent clear so that future changes are not surprising, putting file in the message name clearly sends an intent that other file functionality beyond << might be used in the future. (Of course, using other file functionality means that we might have to rewrite the test.)

So, duck typing is convenient for testing, but what about in the body of applications themselves? Well, it turns out that the same feature that made the tests easy in the previous example also makes it easy to write flexible application code.

In fact, Dave once had an interesting experience where duck typing dug him (and a client) out of a hole. He'd written a large Ruby-based web application that (among other things) kept a database table full of details of participants in a competition. The system provided a comma-separated value (CSV) download capability so administrators could import this information into their local spreadsheets.